

Claude Code Strategy

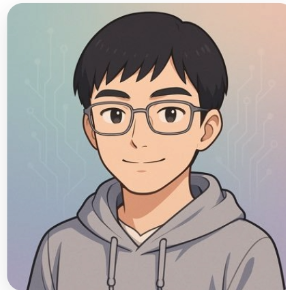
大規模実装を Claude Code に任せるには？

— Plan + Tasks で精度を上げる4つのコツ —

自己紹介

スナガク（本名：馬場俊輔・株式会社 無式）

- ソフトウェアエンジニア
- 趣味: 個人開発 サウナ
- 最近は Claude Code にハマリ中



個人開発

- AIに気軽に恋愛相談できるアプリ **Lovady**
- 「今やるべきこと」を明確にするアプリ **FocusOne**



<https://lovady.app/>



<https://focusone.app/>

目次

よくある失敗パターン

1. Dig コマンドを使い、実装に必要な情報を揃える

2. AI の別モデルを使いレビューさせる

3. Plan 作成時に単体テストの計画も立てる

4. Tasks で ToDo に落として正しく実装させる

Planモード利用時の追加 Tips

よくある失敗パターン

1. タスクの詳細情報がなく、ざっくり指示になっている

Plan を立てる段階で徹底して AI と会話し壁打ちして、実装する上で必要な情報を全て与える

2. 異常系のケース漏れなどの抜け漏れが出やすい

Plan 作成の段階で「作成予定のクラス・関数の責務」と「単体テストのテストケース」を決める

3. Plan を実装する過程で、一部が抜け落ちることがある

Tasks を作成することで、実装の順序を定義して正しい順番で漏れなく実装する

これらの問題を解消するための方法をお伝えします

1. Dig コマンドを使い 実装に必要な情報を揃える

Dig コマンドを使い実装に必要な情報を揃える

@kuu さんが作成されたスラッシュコマンド

プラグインとしてインストール可能

プラン作成時、足りない情報を質問してくれる

自分の考慮できてなかった観点も、一緒に可視化できる

実装に必要な情報を Claude が揃えてくれる

こちらは質問に答えるだけで、必要な情報を渡せる

```
/dig:dig
Claude の Hook スクリプトを Python で書くときに使える型定義ライブラリ
cc-hooks-py を作りたい。
やりたいこと:
各 Hook イベント (PreToolUse, PostToolUse, UserPromptSubmit, Notification, Stop
等) の stdin ペイロードと stdout レスポンスに対応する Python の型 (Pydantic
モデル or TypedDict) を提供する
from cc_hooks import PreToolUseInput, PreToolUseOutput のように import
して使える
IDE の補完と型チェックが効くようにする
Hook 作成時のボイラプレート (stdin 読み取り → パース → 処理 → JSON
出力) を簡単にするヘルパーも欲しい

• Let me first check the current project state and any existing plan files.
  Recalled 1 memory, searched memories, searched for 2 patterns, read 2 files
  (ctrl+o to expand)

• Let me also check for any existing Claude Code hooks and CLAUDE.md for
  context.
  Read 1 file (ctrl+o to expand)

• Task(Research Claude Code hooks spec)
  Done (7 tool uses · 69.5k tokens · 1m 53s)
  (ctrl+o to expand)

• I now have a comprehensive understanding of the Claude Code hooks system (15
  events, detailed stdin/stdout schemas). The project is brand new (just LICENSE
  + git). Let me start the clarification interview.
```

☐ Type system ☐ Python ver ☐ Helper API ☐ Scope ☒ Submit →

Which type system should the library use for defining hook payloads?

1. Pydantic v2 models (Recommended)
Pros: Runtime validation, JSON serialization built-in, excellent IDE support, .model_dump_json(). Cons: Adds pydantic as a hard dependency (~2MB), heavier for simple scripts.
2. TypedDict + dataclasses hybrid
Pros: Zero external dependencies, stdlib only, lightweight. Cons: No runtime validation, need manual JSON serialization, less IDE magic.
3. Pydantic with TypedDict fallback
Pros: Best of both worlds, pydantic optional. Cons: Two code paths to maintain, more complex internals, potential inconsistency.
4. Type something.
5. Chat about this

2. AI の別モデルを使い レビューさせる

AI の別モデルを使ったレビュー

複数の AI Agnet を立ち上げてレビューさせる

Codex ・ Gemini ・ Claude Code を立ち上げてレビュー

抜け漏れていた観点に気づける

プラン作成時に抜けていた観点やエッジケースを指摘してくれる

Plan 作成時に加え、実装後のレビューでも有効

```
1 # カスタムペルソナ機能 実装計画レビュー
2
3 **対象計画:** `docs/plans/wondrous-strolling-zephyr.md`
4 **レビュー日:** 2026-02-06
5 **レビューア:** Claude Agent
6
7 ---
8
9 ## 計画の妥当性
10
11 ### 全体評価
12 計画は要件に対して適切に設計されており、既存アーキテクチャとの整合性が高い。特に以下の点が優れている:
13
14 1. **統一ペルソナサービスの抽出 (Step 2):** `consult.service.ts` と `auto-conversation.service.ts` に重複して
15 いる `loadPersona()` / `loadPersonas()` / `loadAllPersonaMeta()` を統一する方針は正しい。実際にコードを確認する
16 と、3箇所 (consult.service.ts:32-37, auto-conversation.service.ts:47-103, routes/persona.ts:25-61) に同一ロ
17 ジックが存在しており、DRY原則に沿った適切なリファクタリングである。
18
19 2. **バリデーション戦略 (Step 3):** `cp_` prefix で構造チェックのみ行い、存在確認はサービス層で担保するアプローチは、
20 同期開放の `isValidPersonaId()` を非同期にせず維持できるため妥当。既存の `validateAutoConversationConfig()` にも
21 波及影響が少ない。
22
23 3. **データベース設計 (Step 1):** グローバル (プロジェクト横断) としてシンプルなテーブル設計。既存の `settings` テーブル
24 と同様のパターンで、`projectId` を持たない設計は要件に合致。
25
26 4. **実装順序:** バックエンド → フロントエンドの依存関係順で適切。各ステップ間の依存も正しく把握されている。
27
28 ### 技術選定
29 - Drizzle ORM + SQLite: 既存パターン踏襲で問題なし
30 - TanStack Query mutations: `use-projects.ts` パターン参照で一貫性あり
31 - i18n (next-intl): 既存パターン踏襲
32
33 ---
34
35 ## 抜け漏れ・懸念点
36
37 ### 1. SSE イベントでの `emoji` フィールド伝播が未記載
38
39 **重要度: 高**
40
41 `consult.service.ts` の `createSSECallbacks()` (line 309-316) では `persona_start` イベントで `icon` フィー
42 ルドを送信している:
43 ```typescript
44 await writeSSE['event': persona_start](data: ${JSON.stringify({
45   personaId: persona.id,
46   personaName: persona.name,
```


3. Plan 作成時に 単体テストの計画も立てる

Plan 作成時に単体テストの計画も立てる

Plan モードの動作確認は正常系だけのことが多い

事前にテストケースを定義することで、実装の精度を高められる

関数やクラスの責務・Input/Outputを明確化

エッジケースでの振る舞いを制御できる

AI 自身がテストを実行し、不具合を検出できる

人が確認する手間を減らしつつ、実装漏れを防げる

```
## 仕様駆動設計: fs + SQLite + R2 ストレージ統合
| SQL to Chat-DBK to generate
### ドメインモデル

#### Entities

| Entity | 識別子 | 主要属性 | 振る舞い |
|-----|-----|-----|-----|
| StoredFile | id (UUID v4) | sessionId, projectId, category, filename, mimeType, contentType, storageKey, size, metadata | getUrl(), isText(), isR2() |
| Message (既存拡張) | id | imageFileIds (JSON text) | getImageFiles() |

#### Value Objects

| VO | 属性 | 不変条件 |
|-----|-----|-----|
| FileCategory | 'minutes' \| 'ui_mock' \| 'upload' | 3種のみ |
| StorageKey | string | 'uploads/{sessionId}/{uuid}.{ext}' 形式 |
| FileUrl | string | 環境依存、動的生成 |

#### クラス設計

#### 1. StorageStrategy (インターフェース)

**責任:** ファイル保存・削除の抽象化

| メソッド | 入力 | 出力 | 副作用 | 雑粋? |
|-----|-----|-----|-----|-----|
| save | {projectId, sessionId, file: UploadFile} | Promise<SaveResult> | ファイル書き込 + DB INSERT | ✖ |
| deleteBySession | {sessionId} | Promise<void> | ファイル削除 + DB DELETE | ✖ |

```typescript
interface SaveResult {
 fileId: string; // stored_files.id
 storageKey: string; // R2 の場合のみ
}

interface StorageStrategy {
 save(projectId: string, sessionId: string, file: UploadFile): Promise<SaveResult>;
 deleteBySession(sessionId: string): Promise<void>;
}
```

## 4. Tasks で ToDo に落とし 正しく実装させる

# Tasks を使って ToDo に落とし 正しく実装させる

## タスクリストを作成して 進捗と残作業を可視化できる

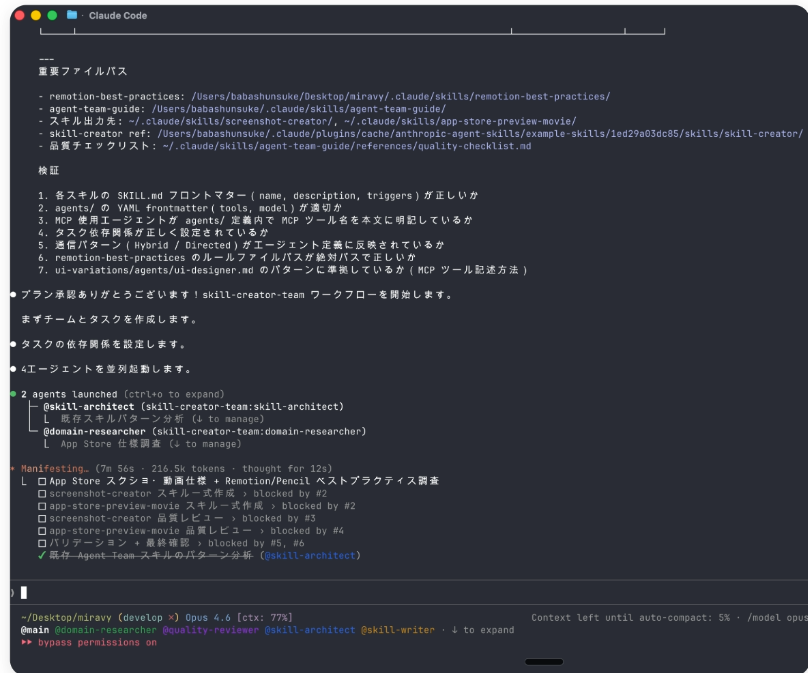
「タスクリストを作って」と依頼するだけで作成してくれる

## タスク毎に依存関係を設定できる

実装→動作確認など、取り組むタスクの順序を決められる

## Agent Teams で 複数タスクを 同時並行で進められる

速度は上がるが、トークン消費量が多くなるので注意



```

重要ファイルパス
- remotion-best-practices: /Users/babashunsuke/Desktop/miravy/claude/skills/remotion-best-practices/
- agent-team-guide: /Users/babashunsuke/claude/skills/agent-team-guide/
- スキル出力先: ~/.claude/skills/screenshot-creator/, ~/.claude/skills/app-store-preview-movie/
- skill-creator ref: /Users/babashunsuke/claude/plugins/cache/anthropic-agent-skills/example-skills/1ed29a03dc85/skills/skill-creator/
- 品質チェックリスト: ~/.claude/skills/agent-team-guide/references/quality-checklist.md

検証
1. 各スキルの SKILL.md フロントマター (name, description, triggers) が正しいか
2. agents/ の YAML frontmatter (tools, model) が適切か
3. MCP 使用エージェントが agents/ 定義内で MCP ツール名を本文に明記しているか
4. タスク依存関係が正しく設定されているか
5. 通信パターン (Hybrid / Directed) がエージェント定義に反映されているか
6. remotion-best-practices のルールファイルパスが絶対パスで正しいか
7. ui-variations/agents/ui-designer.md のパターンに準拠しているか (MCP ツール記述方法)

● ブラン承認ありがとうございます! skill-creator-team ワークフローを開始します。

まずチームとタスクを作成します。

● タスクの依存関係を設定します。

● エージェントを並列起動します。

2 agents launched (ctrl+o to expand)
├─ @skill-architect (skill-creator-team:skill-architect)
│ └─ 既存スキルパターン分析 (do to manage)
│ └─ App Store 仕様調査 (do to manage)
├─ @domain-researcher (skill-creator-team:domain-researcher)
│ └─ App Store 仕様調査 (do to manage)
└─ App Store 仕様調査 (do to manage)

+ Manifesting... (7s 56s - 216.5k tokens - thought for 12s)
├─ ☐ App Store スクショ・動画仕様 + Remotion/Pencil ベストプラクティス調査
├─ ☐ screenshot-creator スキル一式作成 > blocked by #2
├─ ☐ app-store-preview-movie スキル一式作成 > blocked by #2
├─ ☐ screenshot-creator 品質レビュー > blocked by #3
├─ ☐ app-store-preview-movie 品質レビュー > blocked by #4
├─ ☐ バリエーション・最終確認 > blocked by #5, #6
└─ ☒ 既存-Agent-Team-スキルのパターン分析: (@skill-architect)

~/Desktop/miravy (develop ×) 0pus 4.6 [ctx: 77k]
Claudein @domain-researcher @quality-reviewer @skill-architect @skill-writer · do to expand
→ bypass permissions on
```

# Planモード利用時の追加 Tips

# Planモード利用時の追加 Tips

## Plan ファイルの保存ディレクトリ先を変更する

plansDirectory オプションを設定すると、計画書の保存先を指定できる

## Plan 作成時は 必要に応じて随時 Compact を行う

作業の途中で Compaction が走ると、予期せぬ挙動になりやすい

## StatusLine で Context 消費量を可視化する

Compact が必要なタイミングを見極められる

```
"plansDirectory": "./docs/plans",
```

```
~/Desktop/article_lt/LT
(feat/add-gen-ai-article ×)
Opus 4.6 [ctx: 41%]
```

# まとめ

1. Dig コマンドを使い、実装に必要な情報を揃える
2. AI の別モデルでレビューさせて、観点の抜け漏れを防ぐ
3. 単体テストを Skills 化して、エッジケースを制御する
4. Tasks で ToDo に落として、正しい順番で実装させる

ご清聴ありがとうございました！



X: @suna\_gaku



GitHub: claude-code-plugin



@kuuさん: claude-code